

Cluster of Clusters

Blake Muxlow, Tahseen Hussain, Afraz Hameed, Felicia Tao, Andre Chavez
CSE 145/237D, 11.06.2026

1. Project Charter

1.1. Project Overview

The project aims to find the best way to scale large numbers of phones in a way that allows them to work together efficiently. The Junkyard Project has already done some research with regard to the scalability of individual clusters and has found that having a single large cluster with thousands of phones reduces performance. Instead, we plan to create multiple smaller groups of phones which can efficiently work together but avoid performance issues that occur at larger sizes. The goal is to provide clear documentation along with performance metrics which indicate when certain layouts are suitable as well as detailing how to scale from a small number of phones to potentially thousands. We aim to track how certain elements of the cluster layout, such as the number of phones per cluster, or the number of clusters impact the performance of the system. Throughout the quarter, we will be testing these various configurations to find their benefits and tradeoffs as well as clearly outlining when it is reasonable to split a single cluster into multiple of them. Additionally, we aim to use a tool called Karmada, which is meant for managing multiple clusters to create a setup where one cluster manages various sub-clusters of phones which we will outline in more detail below.

1.2. Project Approach

The project approach is to build a federated Kubernetes platform and use it to experimentally study when a single cluster should be split into multiple clusters. We

will first learn and prototype with Kubernetes, Kind, Karmada, and Admiralty, using Kind to quickly create and compare different topologies such as one large cluster, several medium clusters, and many small clusters managed by a super-cluster. We will measure tradeoffs in scalability, manageability, performance, and fault tolerance as we vary cluster size and number. Afterward, we will port the most promising designs to the physical Pixel hardware and integrate them with DSMLP through Admiralty. Based on these experiments, we will produce practical recommendations for how to structure a cluster-of-clusters system and clearly document the platform for future continuation.

To carry this out, we will structure the project in phases. In the first phase, we will focus on building a reliable local prototype environment in Kind to rapidly deploy, tear down, and reconfigure clusters without depending on limited physical hardware. This will let us test the core functionality of Karmada to coordinate multiple clusters and Admiralty as a bridge for access and workloads transitioning across environments. Once this base system is working, we will design controlled experiments that vary the number of clusters, number of nodes per clusters, and the responsibilities assigned to the super-cluster.

In the second phase, we will validate the most promising configurations on the real Pixel clusters to see how well our findings transfer from emulation to hardware. This step is important because real devices introduce practical constraints such as limited resources, network variability, and deployment complexity that might not fully appear in Kind. We will integrate the platform with DSMLP through Admiralty to evaluate how well users and workloads can function across cluster boundaries in realistic scenarios. By combining fast local prototyping with targeted hardware validation, we aim to produce a functioning Kubernetes platform and a set of concrete guidelines for deciding when a single cluster should become many.

1.3. Minimal Viable Product

The MVP for this project is a locally simulated cluster of cluster environment running on KIND and managed by Karmada, capable of deploying workloads across multiple sub clusters. At this stage, the system should support basic cross cluster communication, resource monitoring via Prometheus & Grafana, and a workload process to simulate user activity. A goal here is to establish the core federation architecture and provide an initial set of performance baselines across at least two or different topologies. Hardware integration and automation pipelines come in later deliverables, the MVP is focused on demonstrating that the fundamental approach is viable and gives us concrete data to guide the rest of the project.

A very key component of the MVP is the Karmada control plane, which is the central manager for all of our sub clusters. Karmada must be able to successfully propagate workloads across clusters, enforce scheduling policies, and report cluster metrics back via our monitoring pipeline. In the MVP stage, we are not targeting a fully optimized setup, but feasibility and performance measurements. We will be using KIND to spin up these clusters locally, allowing rapid development on different configurations without physical hardware.

The workload simulation is another major component of the MVP. This script will generate user activity across the cluster, such as student workloads, pod deployment, and cluster communication, in order to stress test our system under realistic conditions. This script should be configurable enough to adjust load intensity across clusters, allowing us to run series of controlled experiments. We aim to run two or three distinct topologies, such as a single large cluster, several medium sized ones, and many small clusters with results documented and compared.

Resource monitoring is another key piece of the MVP. We plan to use Prometheus for data collection and Grafana for visualization. We aim to build a dashboard capable of displaying key metrics such as CPU usage, memory consumption, network traffic, and node count. These metrics are essential for comparing topologies in a meaningful way. Without reliable data collection in place early on, later experimentation would lack the observability needed to draw any meaningful conclusions about when and why clusters should be split

Together, these components form a working simulation pipeline that can validate our core technical approach of the project. The MVP will serve as a functional proof of concept that demonstrates federation, monitoring, and workload simulation all working together. Achieving these milestones will demonstrate in the feasibility of a hardware port later on in the Quarter.

Beyond the MVP, we have two primary stretch goals. The first is porting the cluster of clusters environment to physical Pixel Hardware. While the MVP focuses on Kubernetes feasibility through local KIND simulations, deploying the same topology and performance validation on real pixel nodes will provide a more meaningful set of metrics that future teams can use. A second stretch goal is CI/CD pipelines to automate the process of topology deployment, workload processes, and data collection. We plan to use an automated pipeline called ArgoCD to enable our team to run a higher volume of tests. This would also help generate a reusable testing framework for future teams.

1.4. Constraints, Risk, Feasibility

A primary constraint of this project is the complexity of integrating multiple independently managed systems under a federated platform. Karmada and Admiralty are non-trivial tools that require a solid understanding of core Kubernetes concepts before meaningful configurations can begin. This introduces risk early in the timeline, as weeks spent on foundational learning shorten the time available for experimentation. We aim to address this by front loading learning in Weeks 1-4 using KIND (Kubernetes in Docker), a lightweight tool that allows us to simulate full multi-cluster environments locally without requiring the physical hardware for it. This enables experimentation before full system understanding is actually achieved.

A second concern is integration with DSMLP which is an externally managed system outside of our direct control. Network policies, authentication restrictions, or administrative barriers imposed by any IT service in control could limit what Admiralty is able to federate across. Since this integration cannot be fully prototyped locally, issues may only surface late in the project timeline. To mitigate this risk, communication with DSMLP management early to clear any logistical/bureaucratic hurdles is important.

Finally, porting a working KIND prototype to real Pixel hardware introduces risk around architectural compatibility. The prototypes will most certainly not have a clean translation in terms of configuration, and some manual reconfiguration of networking interfaces, storage drivers, and node labels should be expected during the hardware transition phase. Securing early access to at least one Pixel node for compatibility validation before full deployment will help surface translation issues well ahead of the final project deadline, rather than encountering them all at once during the latter weeks of development. Another strategy is to maintain a separate hardware focused KIND configuration that is incrementally adapted in parallel with our local instances.

Overall, the risks are manageable given proactive mitigation strategies at each stage. By front loading our foundational learning, maintaining early communication with DSMLP administrators, and validating hardware compatibility well before the final deadline, we are well positioned to address challenges as they arise throughout the quarter.

2. Group Management

Management roles are split between project managers who set goals, handle timelines, and perform regular check-ins with the rest of the group to make sure

that they are adhering to said goals. We will all be working on development and building the project together.

Decisions will be broken up into greater project and sub-team decisions. The former will be a vote and involve an informed decision of each member rather than one single facilitator. However, in the case of conflict/tie, there is a dedicated tie-breaker who will make the decision. The latter will be decided within each sub-team and escalated to the tie-breaker if needed.

We will be communicating via the dedicated 145/237D #junkyard-cluster-of-clusters discord chat. We will also be utilizing GitHub functionality for issues, commits, etc.

We will discuss with our team members via discord or in class when there is a scheduling conflict of a singular member. As for our timeline and schedule, we will use the Gantt chart and GitHub issues to ensure that we are on schedule. When there are delays, we will reallocate tasks to different team members or reduce the scope of non-critical features if needed.

The entire team is responsible for all deliverables and individual milestones will be decided by the Gantt chart or on an “as-needed” basis.

3. Project Development

We have broken up our development roles into teams, ie node/cluster management, data collection and visualization, research and documentation.

Most of the tools we are using are open source and free, including Kubernetes, Kind, Karmada, Admiralty, Grafana, etc. The main cost of the project involves any resources needed for testing. However, we can do most of the development and experimentation in Kind on existing machines, and then validate key results on the existing Pixel clusters and through DSMLP.

If we are ever going to need more compute resources, we will first use all university resources accessible to us. If that is not sufficient, we will look into free resources from other providers, so we don't expect to have extra costs.

We will test in two stages. First, we will use Kind to do rapid functional and experimental testing. This includes verifying the clusters can be created and joined to Karmada correctly, that workloads can be deployed across different topologies, and that Admiralty can support federation between test environment and DSMLP.

Second, we will validate the most promising results on the actual Pixel hardware. We will test constraints that do not fully appear in local emulation like lower resources, less stable networking, and more complicated deployment conditions.

We will document the project continuously rather than waiting until the very end. This includes maintaining a shared project repository with a clear README, setup instructions, dependency lists, relevant images/diagrams, and commands for reproducing the environment.

We will also document each topology we test, the purpose of each experiment, metrics collected, and any issues or blockers we encounter. This will make it easier for future teams to continue the project without needing to rediscover tedious setup details.

By the final week, we will consolidate all of this into a polished handoff package including installation and deployment instructions, experiment methodology, results summaries, lessons learned, and recommendations for future collaborators on the project. We plan to document which aspects were validated only in Kind and which were validated on real hardware.

4. Project Milestones & Schedule

4.1. Deliverable 1: Project Specification and Environment Setup

- Dependencies: None
- Description: Define project expectations and scope, finish projects specification, and set up development environment with needed infrastructure and tools
- Milestones:
 - Complete Gantt Chart
 - Complete Project Specification
 - Learn technical stack (Docker, Kubernetes, Karmada, Kind, etc.)
 - Install all required dependencies

4.2. Deliverable 2: Simulated Cluster of Cluster Environment and Initial Analysis (MVP)

- Dependencies: Deliverable 1
- Description: Establish a working local simulation of “Cluster of Clusters” using Kind, managed by Karmada, with data collection and cross-cluster routing
- Milestones:
 - Use Docker, Kubernetes, and Kind to create local worker environments
 - Use Karmada to deploy a control plane and manage the local clusters

- Configure Admiralty/Karmada to simulate DSMLP federation
- Use Prometheus and Grafana to collect and display resource usage statistics
- Create deployment workload script to simulate user activity
- Research and test baseline topologies and document performance
- Prepare Oral Project Update

4.3. Deliverable 3: Automation Pipeline and Deeper Analysis

- Dependencies: Deliverable 2
- Description: Create an automated pipeline for testing new topologies and collecting data, along with diving deeper into the rules for splitting clusters
- Milestones:
 - Review feedback from update and adjust schedule as needed
 - Use Ansible/Terraform to write automated pipelines from topology creation to data collection
 - Improve data collection and visualization dashboard
 - Run more experiments and analyze data to create a set of rules for when clusters should be split
 - Find the maximum number of clusters that can be handled by Kubernetes and Karmada through load testing

4.4. Deliverable 4: Hardware Setup

- Dependencies: Deliverable 3
- Description: Begin setting up and configuring the physical hardware (Pixel phones) to connect them to Karmada and create a super cluster
- Milestones:
 - Configure OS and local network on Pixel phones
 - Deploy lightweight version of Kubernetes
 - Connect all nodes to Karmada control plane
 - Apply Admiralty/Karmada access policies
 - Modify data collection to account for hardware constraints

4.5. Deliverable 5: Hardware Experiments and Testing

- Dependencies: Deliverable 4
- Description: Run topologies that do well in simulation on the physical hardware and measure performance, along with testing overall system reliability
- Milestones:
 - Run baseline testing to ensure hardware works as expected
 - Execute best topologies found so far in simulation
 - Execute hardware stress testing and high availability testing
 - Compare physical hardware data with simulation data
 - Test on homogeneous vs heterogeneous mixture of hardware

- Finalize decision making rules for when one cluster should become many

4.6. Deliverable 6: Final Delivery/Presentation

- Dependencies: Deliverable 5
- Description: Gather all data, documentation/reports, visualizations, and code to create a final presentation
- Milestones:
 - Clean up GitHub repository and ensure proper documentation
 - Complete project webpresence detailing architecture and how to reproduce results
 - Complete final video, report, and presentation

For the full details about schedule and task delegation, refer to our [Gantt Chart](#).